

13-07-26

Single linked list

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node
```

```
{
    int data;
    struct Node * next;
```

```
};
```

```
struct Node * head = NULL;
```

// Insert at begin

```
void insertBegin (int value)
```

```
{
    struct Node * newNode = (struct Node *) malloc (sizeof (struct Node));
    newNode->data = value;
    newNode->next = head;
    head = newNode;
```

```
}
```

// Insert at end

```
void insertEnd (int value)
```

```
{
    struct Node * newNode = (struct Node *) malloc (sizeof (struct Node));
```

```
    struct Node * temp;
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    if (head == NULL)
```

```
    {
        head = newNode;
```

```
        return;
```

```
    }
```

```
    temp = head;
```

```
    while (temp->next != NULL)
```

```
        temp = temp->next;
```

```
    temp->next = newNode;
```

```
}
```

// Delete from begin
void deleteBegin()

if (head == NULL)

{
printf("List is empty.\n");

return;

}

struct node *temp = head;

head = head->next;

printf("Deleted element: %d\n", temp->data);

free(temp);

}

// Delete from End

void deleteEnd()

if (head == NULL)

{
printf("List is empty.\n");

return;

}

if (head->next == NULL)

{
printf("Deleted element: %d\n", head->data);

free(head);

head = NULL;

return;

}

struct node *temp = head;

while (temp->next->next != NULL)

temp = temp->next;

printf("Deleted element: %d\n", temp->next->data);

free(temp->next);

temp->next = NULL;

}

```

// display
void display ()
{
    struct Node *temp = head;
    if (head == NULL)
    {
        printf ("List is empty.\n");
        return;
    }
    printf ("Linked list : ");
    while (temp != NULL)
    {
        printf ("%d -> ", temp->data);
        temp = temp->next;
    }
    printf ("NULL\n");
}

```

```

int main ()
{
    int choice, value;
    while (1)
    {
        printf ("n-- Singly linked List --n\n");
        printf ("1. Insert at Begin\n");
        printf ("2. Insert at End\n");
        printf ("3. Delete from Begin\n");
        printf ("4. Delete from End\n");
        printf ("5. Display\n");
        printf ("6. Exit\n");
        printf ("Enter your choice : ");
        scanf ("%d", &choice);
        switch (choice)

```

```

        {
            case 1:
                printf ("Enter value : ");
                scanf ("%d", &value);
                insert Begin (value);

```

case 2:

```
printf ("Enter value: ");  
scanf ("%d", &value);  
insertEnd (value);  
break;
```

case 3:

```
deleteBegin();  
break;
```

case 4:

```
deleteEnd();  
break;
```

case 5:

```
display();  
break;
```

case 6:

```
exit(0);
```

default:

```
printf ("Invalid choice!\n");
```

```
}  
return 0;
```

```
}
```

Output

-- singly linked list --

1. Insert at Begin
2. Insert at End
3. Delete from Begin
4. Delete from End
5. Display
6. exit

enter your choice: 1

enter value : 2, 4, 6

enter your choice: 2

enter value : 8, 10

enter your choice : 3

enter value : 6

enter your choice : 4

enter value : 10

enter your choice : 5

Linked list : 4 → 2 → 8 → null

```
# include <stdio.h>
# include <stdlib.h>
struct node {
    int data;
    struct node * next;
};
struct node * head=NULL;
// insert at begin
void insertBegin(int value)
{
    struct node * newNode=(struct node *) malloc (sizeof(struct node));
    newNode->data=value;
    newNode->next=head;
    head=newNode;
}
//insert at end
void insertEnd(int value)
{
    struct node * newNode=(struct node *)malloc (sizeof(struct node));
    struct node * temp;
    newNode->data=value;
    newNode->next=NULL;
    if (head==NULL)
    {
        head=newNode;
        return;
    }
    temp=head;
    while (temp->next!=NULL)
    temp=temp->next;
    temp->next=newNode;
}
```

```

// delete from begin
void deleteBegin()
{
    if(head==NULL)
    {
        printf("list is empty.\n");
        return;
    }
    struct node * temp=head;
    head=head->next;
    printf("deleted element :%d\n",temp->data);
    free(temp);
}

// delete from end
void deleteEnd()
{
    if(head==NULL)
    {
        printf("list is empty.\n");
        return;
    }
    if(head->next==NULL)
    {
        printf("deleted element :%d\n",head->data);
        free(head);
        head=NULL;
        return;
    }
    struct node * temp=head;
    while(temp->next->next!=NULL)
    temp=temp->next;
    printf("deleted element :%d\n",temp->next->data);
    free(temp->next);
    temp->next=NULL;
}

```

```

// display
void display()
{
    struct node * temp=head;
    if(head==NULL)
    {
        printf("list is empty.\n");
        return;
    }
    printf("linked list:");
    while(temp!=NULL)
    {
        printf("%d->",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}

int main()
{
    int choice,value;
    while(1)
    {
        printf("\n--singly linked list--\n");
        printf("1.Insert at Begin\n");
        printf("2.Insert at End\n");
        printf("3.Delete from Begin\n");
        printf("4.Delete from End\n");
        printf("5.Display\n");
        printf("6. exit\n");
        printf("enter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {

```

```
switch(choice)
{
    case 1 :
        printf("enter value:");
        scanf("%d",&value);
        insertBegin(value);
        break;

    case 2 :
        printf("enter value:");
        scanf("%d",&value);
        insertEnd(value);
        break;

    case 3 :
        deleteBegin();
        break;

    case 4 :
        deleteEnd();
        break;

    case 5 :
        display();
        break;

    case 6 :
        exit(0);

    default :
        printf("Invalid choice\n");
}

return 0;
```

```
}
```

--singly linked list--

- 1.Insert at Begin
- 2.Insert at End
- 3.Delete from Begin
- 4.Delete from End
- 5.Display
6. exit

enter your choice:1

enter value:2

--singly linked list--

- 1.Insert at Begin
- 2.Insert at End
- 3.Delete from Begin
- 4.Delete from End
- 5.Display
6. exit

enter your choice:1

enter value:4

--singly linked list--

- 1.Insert at Begin
- 2.Insert at End
- 3.Delete from Begin
- 4.Delete from End
- 5.Display
6. exit

enter your choice:1

enter value:6

```
--singly linked list--  
1.Insert at Begin  
2.Insert at End  
3.Delete from Begin  
4.Delete from End  
5.Display  
6. exit  
enter your choice:2  
enter value:8
```

```
--singly linked list--  
1.Insert at Begin  
2.Insert at End  
3.Delete from Begin  
4.Delete from End  
5.Display  
6. exit  
enter your choice:2  
enter value:10
```

```
--singly linked list--  
1.Insert at Begin  
2.Insert at End  
3.Delete from Begin  
4.Delete from End  
5.Display  
6. exit  
enter your choice:3  
deleted element :6
```

```
--singly linked list--  
1.Insert at Begin  
2.Insert at End  
3.Delete from Begin  
4.Delete from End  
5.Display  
6. exit  
enter your choice:4  
deleted element :10
```

```
--singly linked list--  
1.Insert at Begin  
2.Insert at End  
3.Delete from Begin  
4.Delete from End  
5.Display  
6. exit  
enter your choice:5  
linked list:4->2->8->NULL
```

```
--singly linked list--  
1.Insert at Begin  
2.Insert at End  
3.Delete from Begin  
4.Delete from End  
5.Display  
6. exit
```